

Lab 4: Parallel task decomposition implementation and analysis

Original tile iterative task decomposition

Section	Description
Task Decomposition	Original tile iterative task decomposition
Code	The full code is in the file: "mandel-omp-original.c" The most important parts of the code are: · First we code the <code>#pragma omp parallel</code> and <code>#pragma omp single</code> to open a parallel region. · Secondly, we don't need to put any <code>firstprivate</code> in the declaration of the task, because as the variables have been declared in the parallel region each task will have it's own copy. · Later we put two <code>#pragma atomic</code>: <code>#pragma omp atomic</code> <code>histogram[M[y][x] - 1] += (TILE*TILE);</code> <code>#pragma omp atomic</code> <code>histogram[M[py][px] - 1]++;</code> We did that because we need to prevent a "Data Race" between tasks. · To finish we put two more <code>#pragma omp critical</code>: <code>#pragma omp critical</code> <code>{</code> <code>XSetForeground(display, gc, color);</code> <code>XDrawPoint(display, win, gc, px, py);</code> <code>}</code> and later there is another below. We have done this for the same reason as we did with atomic, but we do critical because those lines aren't an arithmetic operation like <code>+, -, *, /</code>.
Modelfactor analysis	· First table: The first table shows that the efficiency decreases more and more as the number of threads increases. This is due to the fact

that in a certain point, even if you put more threads, the speed up wouldn't increase. We want to add that the elapsed time remains constant when the number of threads is 8 or more. So we can say that for this program 8 or 12 threads it's enough.

Overview of whole program execution metrics										
Number of threads	1	2	4	8	12	16	20	24	28	32
Elapsed time (sec)	3.52	1.98	1.04	0.74	0.73	0.73	0.73	0.73	0.73	0.73
Speedup	1.00	1.77	3.39	4.73	4.81	4.80	4.80	4.81	4.81	4.80
Efficiency	1.00	0.89	0.85	0.59	0.40	0.30	0.24	0.20	0.17	0.15

Table 1: Analysis done on Tue Apr 14 11:30:58 AM CEST 2026, par1104

• Second table:

Analyzing the second table we can observe that on average the numbers show good results. The parallel fraction of the program is 99.9%, which means that almost everything of the program has been parallelized. But we have to observe that the Global efficiency is decreasing rapidly because of the Load balancing. What's happening is that there is a thread spending a lot of time computing tasks, much more time than the time per thread on average. This tells us that there is load imbalance.

Overview of the Efficiency metrics in parallel fraction, $\phi=99.99\%$										
Number of threads	1	2	4	8	12	16	20	24	28	32
Global efficiency	100.00%	88.75%	84.85%	59.16%	40.06%	30.03%	24.02%	20.04%	17.18%	15.02%
Parallelization strategy efficiency	100.00%	88.84%	84.99%	59.38%	40.30%	30.36%	24.33%	20.32%	17.48%	15.44%
Load balancing	100.00%	88.86%	85.01%	59.41%	40.36%	30.39%	24.37%	20.37%	17.51%	15.49%
In execution efficiency	100.00%	99.98%	99.97%	99.94%	99.87%	99.89%	99.82%	99.78%	99.82%	99.73%
Scalability for computation tasks	100.00%	99.90%	99.84%	99.63%	99.39%	98.92%	98.71%	98.60%	98.29%	97.25%
IPC scalability	100.00%	100.00%	99.97%	99.97%	99.98%	99.97%	99.96%	99.97%	99.96%	99.96%
Instruction scalability	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
Frequency scalability	100.00%	99.90%	99.86%	99.66%	99.41%	98.95%	98.75%	98.63%	98.33%	97.29%

Table 2: Analysis done on Tue Apr 14 11:30:58 AM CEST 2026, par1104

• Third table:

Here we can observe that the *sched Overhead* is always 0.0 because when making the tables the execution does not generate any histogram nor image, so the atomic and the critical are not executed, hence not introducing overhead.

The *synch Overhead* increases as the threads also increase. This is normal because as more threads are, more chances of one thread to wait for the others increase. At the end the overhead by synch is 550%, that means that by 100s of computing there is 500s of overhead, and that is terrible. Both LB decrease as the number of threads increase, this does not mean anything bad, maybe one thread executes one big task while the other does a lot but with a smaller size.

Statistics about explicit tasks in parallel fraction										
Number of threads	1	2	4	8	12	16	20	24	28	32
Number of explicit tasks executed (total)	64.0	64.0	64.0	64.0	64.0	64.0	64.0	64.0	64.0	64.0
LB (number of explicit tasks executed)	1.0	0.94	0.64	0.44	0.31	0.24	0.19	0.16	0.13	0.12
LB (time executing explicit tasks)	1.0	0.89	0.85	0.59	0.4	0.3	0.24	0.2	0.17	0.15
Time per explicit task (average us)	54951.19	55008.19	55034.71	55132.45	55222.17	55439.01	55486.72	55476.6	55542.86	56053.96
Overhead per explicit task (synch %)	0.0	12.55	17.64	68.35	147.99	229.49	311.3	392.79	474.16	550.15
Overhead per explicit task (sched %)	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.01	0.0
Number of taskwait/taskgroup (total)	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

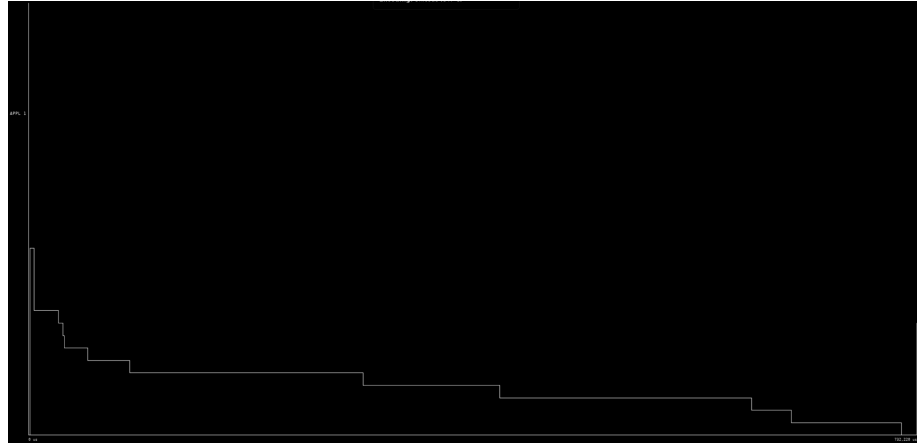
Table 3: Analysis done on Tue Apr 14 11:30:58 AM CEST 2026, par1104

Paraver

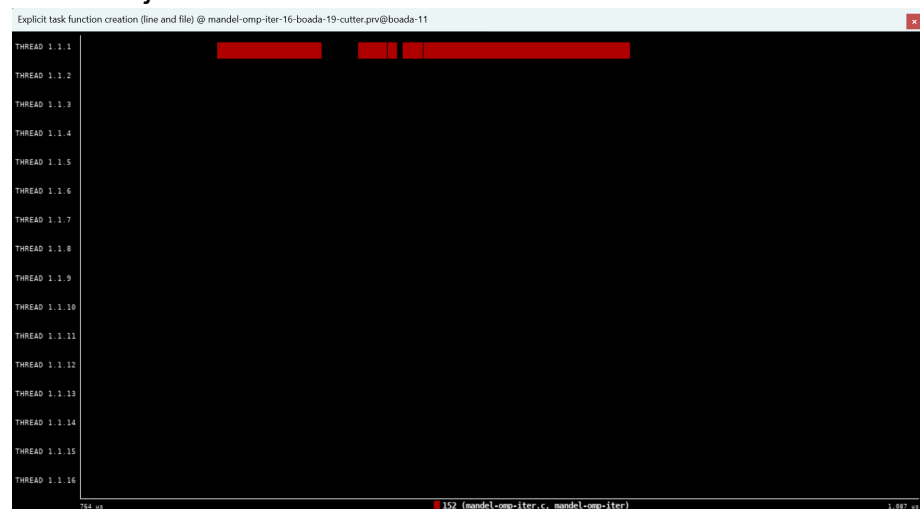
• Hint Useful/Instantaneous Parallelism:

analysis

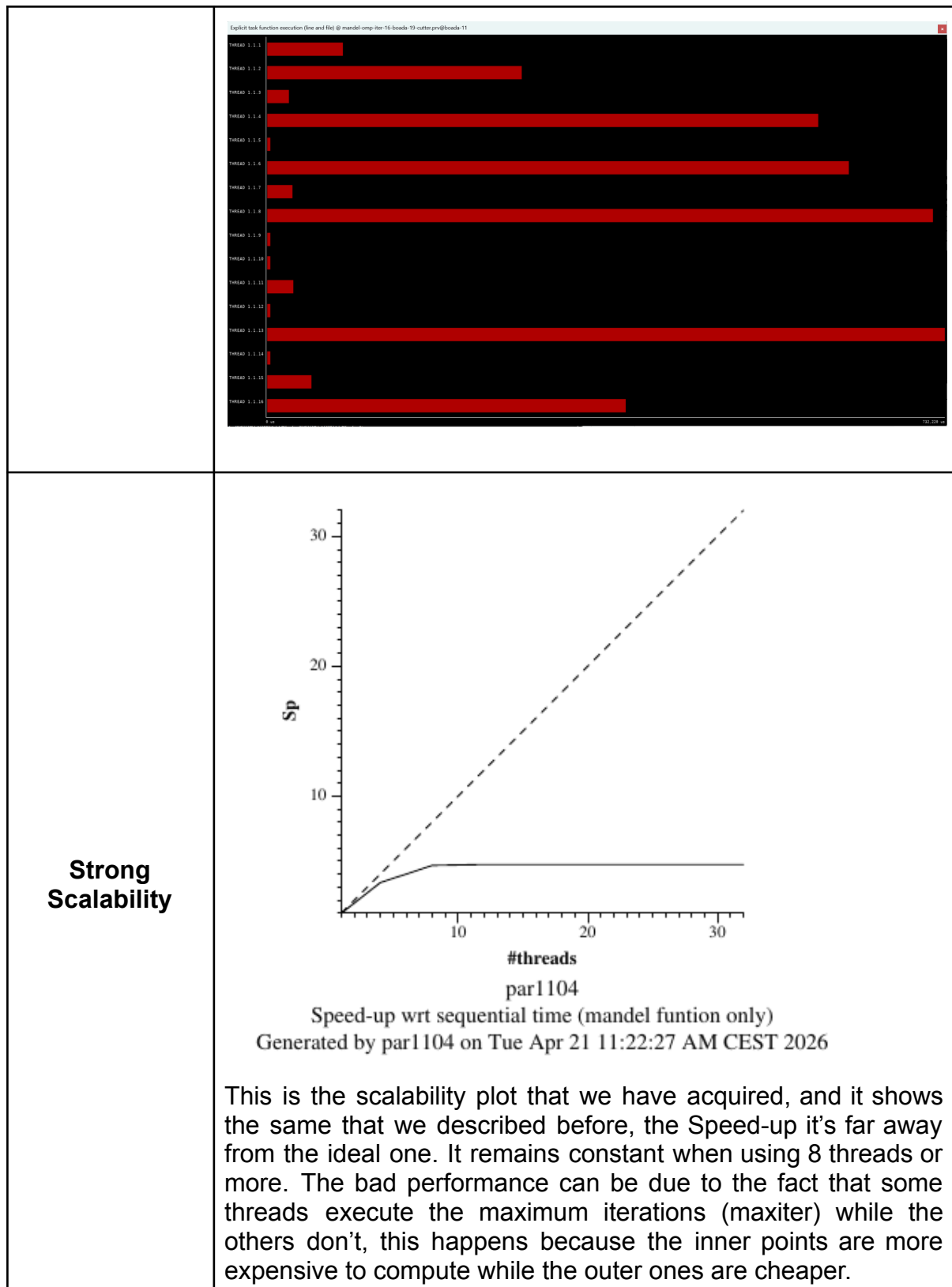
This hint tells us that there is a lot of load imbalance, just as we noticed before. We can deduce this because at the beginning of the program there are a lot of threads doing tasks at the same time, but at the end just a few of them keep working and making tasks.

**· Hint Tasking/explicit tasks functions created:**

This diagram tells us that the master thread creates all the tasks at the beginning of the execution, which is good and the thread does it in just 300 microseconds.

**· Hint Tasking/explicit tasks functions executed:**

The diagram below shows us that the existence of load imbalance is plausible, because of 16 threads available, only a few do work all the time. If we look at the threads, only the 4th, 6th, 8th and 13th do the heavy work, the others do half of it or even almost nothing like the 5th, 9th, 10th or 14th.



Fine grain iterative task decomposition

Section	Description
Task Decomposition	Fine grain iterative task decomposition
Code	The full code is in "mandel-omp-fine-grain.c". To explain here we are going to split the code in 2 parts, the part of the equal and the part of the if. Part of the equal We keep the <code>#pragma omp task</code> at the start like in the original iterative task because the exercise requires it. Later we start coding the new part, we code: <pre>#pragma omp taskgroup task_reduction(&:equal)</pre> We did this because we want to create tasks and when they finish the taskgroup waits, also we did a <code>task_reduction</code> on <code>equal</code> because inside the taskgroup we have <code>equal</code> doing a <code>&</code> operation and we need to avoid a Data Race condition. Inside the taskgroup we create two tasks with <code>in_reduction(&:equal)</code>: • The first task before the first loop to the vertical • The second task before the second loop to the horizontal We split them into two separate tasks to get fine-grain parallelism, so both borders can be computed at the same time. The <code>in_reduction</code> is necessary so each task can safely contribute to the shared <code>equal</code> variable because it's variable do <code>&</code> operation. Part of the if We code one <code>#pragma omp task</code> before the <code>if(equal)</code> to get the same fine-grain structure, so the conditional logic runs as its own task once the taskgroup finishes. Inside the <code>if</code> we keep the same atomic and critical as the original tile: <pre>#pragma omp atomic histogram[M[y][x] - 1] += (TILE*TILE); #pragma omp atomic histogram[M[py][px] - 1]++;</pre> We use atomic because those are simple arithmetic operations on

	a shared array and we need to prevent a Data Race between tasks. Also we put <code>#pragma omp task</code> between the <code>py</code> loops in both branches of the <code>if</code>, so each row of the tile is computed in parallel. Finally we keep the critical sections: <pre>#pragma omp critical { XSetForeground(display, gc, color); XDrawPoint(display, win, gc, px, py); }</pre> We did that for the same reason as we did with <code>atomic</code>, but we use <code>critical</code> because those lines are not arithmetic operations, they are function calls that need to execute as a whole block.																																																																																																																																																																					
Modelfactor analysis	First table: The first table shows that the efficiency decreases a little as the number of threads increases, but it stays quite high, reaching 0.82 at 32 threads. The speedup is calculated as the elapsed time with 1 thread divided by the elapsed time with N threads, and increase very well. This is because the parallel fraction of the program is very high so almost all the work is being done in parallel. The elapsed time keeps going down even if we increase number of threads so we can say this version scales well. <table><tr><th colspan="11">Overview of whole program execution metrics</th></tr><tr><th>Number of threads</th><th>1</th><th>2</th><th>4</th><th>8</th><th>12</th><th>16</th><th>20</th><th>24</th><th>28</th><th>32</th></tr><tr><td>Elapsed time (sec)</td><td>3.53</td><td>1.77</td><td>0.89</td><td>0.45</td><td>0.31</td><td>0.24</td><td>0.20</td><td>0.17</td><td>0.15</td><td>0.13</td></tr><tr><td>Speedup</td><td>1.00</td><td>1.99</td><td>3.95</td><td>7.78</td><td>11.42</td><td>14.76</td><td>18.03</td><td>21.23</td><td>23.74</td><td>26.29</td></tr><tr><td>Efficiency</td><td>1.00</td><td>0.99</td><td>0.99</td><td>0.97</td><td>0.95</td><td>0.92</td><td>0.90</td><td>0.88</td><td>0.85</td><td>0.82</td></tr></table> Table 1: Analysis done on Tue Apr 21 11:40:23 AM CEST 2026, par1119 Second table: Looking at the second table we can see that the global efficiency goes down as the number of threads increases, from 99.94% to 82.32%. The main reason for this is the Load balancing, which drops to 87.42% with 32 threads. This means that one thread is spending much more time computing than the others, and since the Parallelization strategy efficiency is calculated as Load balancing multiplied by In execution efficiency, this pulls the whole Parallelization strategy efficiency down, and with it the Global efficiency. However, the Scalability for computation tasks stays very high at 98.69% with 32 threads, and the others, IPC scalability, Instruction scalability and Frequency scalability, all stay very close to 100%, which means the hardware used in a good way. <table><tr><th colspan="11">Overview of the Efficiency metrics in parallel fraction, $\phi=99.99\%$</th></tr><tr><th>Number of threads</th><th>1</th><th>2</th><th>4</th><th>8</th><th>12</th><th>16</th><th>20</th><th>24</th><th>28</th><th>32</th></tr><tr><td>Global efficiency</td><td>99.94%</td><td>99.36%</td><td>98.82%</td><td>97.27%</td><td>95.26%</td><td>92.30%</td><td>90.25%</td><td>88.61%</td><td>84.92%</td><td>82.32%</td></tr><tr><td>Parallelization strategy efficiency</td><td>99.94%</td><td>99.46%</td><td>98.96%</td><td>97.56%</td><td>95.78%</td><td>93.08%</td><td>91.33%</td><td>89.62%</td><td>86.18%</td><td>83.42%</td></tr><tr><td>Load balancing</td><td>100.00%</td><td>99.93%</td><td>99.76%</td><td>99.18%</td><td>98.20%</td><td>97.42%</td><td>96.60%</td><td>92.87%</td><td>91.40%</td><td>87.42%</td></tr><tr><td>In execution efficiency</td><td>99.94%</td><td>99.53%</td><td>99.20%</td><td>98.36%</td><td>97.54%</td><td>95.55%</td><td>94.54%</td><td>96.50%</td><td>94.28%</td><td>95.43%</td></tr><tr><td>Scalability for computation tasks</td><td>100.00%</td><td>99.90%</td><td>99.86%</td><td>99.71%</td><td>99.46%</td><td>99.15%</td><td>98.82%</td><td>98.87%</td><td>98.54%</td><td>98.69%</td></tr><tr><td>IPC scalability</td><td>100.00%</td><td>99.87%</td><td>99.86%</td><td>99.84%</td><td>99.82%</td><td>99.79%</td><td>99.79%</td><td>99.80%</td><td>99.81%</td><td>99.77%</td></tr><tr><td>Instruction scalability</td><td>100.00%</td><td>100.10%</td><td>100.10%</td><td>100.09%</td><td>100.09%</td><td>100.09%</td><td>100.09%</td><td>100.09%</td><td>100.09%</td><td>100.09%</td></tr><tr><td>Frequency scalability</td><td>100.00%</td><td>99.94%</td><td>99.90%</td><td>99.77%</td><td>99.55%</td><td>99.27%</td><td>98.94%</td><td>98.98%</td><td>98.64%</td><td>98.83%</td></tr></table>	Overview of whole program execution metrics											Number of threads	1	2	4	8	12	16	20	24	28	32	Elapsed time (sec)	3.53	1.77	0.89	0.45	0.31	0.24	0.20	0.17	0.15	0.13	Speedup	1.00	1.99	3.95	7.78	11.42	14.76	18.03	21.23	23.74	26.29	Efficiency	1.00	0.99	0.99	0.97	0.95	0.92	0.90	0.88	0.85	0.82	Overview of the Efficiency metrics in parallel fraction, $\phi=99.99\%$											Number of threads	1	2	4	8	12	16	20	24	28	32	Global efficiency	99.94%	99.36%	98.82%	97.27%	95.26%	92.30%	90.25%	88.61%	84.92%	82.32%	Parallelization strategy efficiency	99.94%	99.46%	98.96%	97.56%	95.78%	93.08%	91.33%	89.62%	86.18%	83.42%	Load balancing	100.00%	99.93%	99.76%	99.18%	98.20%	97.42%	96.60%	92.87%	91.40%	87.42%	In execution efficiency	99.94%	99.53%	99.20%	98.36%	97.54%	95.55%	94.54%	96.50%	94.28%	95.43%	Scalability for computation tasks	100.00%	99.90%	99.86%	99.71%	99.46%	99.15%	98.82%	98.87%	98.54%	98.69%	IPC scalability	100.00%	99.87%	99.86%	99.84%	99.82%	99.79%	99.79%	99.80%	99.81%	99.77%	Instruction scalability	100.00%	100.10%	100.10%	100.09%	100.09%	100.09%	100.09%	100.09%	100.09%	100.09%	Frequency scalability	100.00%	99.94%	99.90%	99.77%	99.55%	99.27%	98.94%	98.98%	98.64%	98.83%
Overview of whole program execution metrics																																																																																																																																																																						
Number of threads	1	2	4	8	12	16	20	24	28	32																																																																																																																																																												
Elapsed time (sec)	3.53	1.77	0.89	0.45	0.31	0.24	0.20	0.17	0.15	0.13																																																																																																																																																												
Speedup	1.00	1.99	3.95	7.78	11.42	14.76	18.03	21.23	23.74	26.29																																																																																																																																																												
Efficiency	1.00	0.99	0.99	0.97	0.95	0.92	0.90	0.88	0.85	0.82																																																																																																																																																												
Overview of the Efficiency metrics in parallel fraction, $\phi=99.99\%$																																																																																																																																																																						
Number of threads	1	2	4	8	12	16	20	24	28	32																																																																																																																																																												
Global efficiency	99.94%	99.36%	98.82%	97.27%	95.26%	92.30%	90.25%	88.61%	84.92%	82.32%																																																																																																																																																												
Parallelization strategy efficiency	99.94%	99.46%	98.96%	97.56%	95.78%	93.08%	91.33%	89.62%	86.18%	83.42%																																																																																																																																																												
Load balancing	100.00%	99.93%	99.76%	99.18%	98.20%	97.42%	96.60%	92.87%	91.40%	87.42%																																																																																																																																																												
In execution efficiency	99.94%	99.53%	99.20%	98.36%	97.54%	95.55%	94.54%	96.50%	94.28%	95.43%																																																																																																																																																												
Scalability for computation tasks	100.00%	99.90%	99.86%	99.71%	99.46%	99.15%	98.82%	98.87%	98.54%	98.69%																																																																																																																																																												
IPC scalability	100.00%	99.87%	99.86%	99.84%	99.82%	99.79%	99.79%	99.80%	99.81%	99.77%																																																																																																																																																												
Instruction scalability	100.00%	100.10%	100.10%	100.09%	100.09%	100.09%	100.09%	100.09%	100.09%	100.09%																																																																																																																																																												
Frequency scalability	100.00%	99.94%	99.90%	99.77%	99.55%	99.27%	98.94%	98.98%	98.64%	98.83%																																																																																																																																																												

Third table: Here we can see that the sched overhead isn't 0, which is different from original version. This is because in this version we use task_reduction and in_reduction for the equal variable, so when the inner tasks finish they need to synchronize and combine their results, and that introduces scheduling overhead. This overhead grows as the threads increase, reaching 7.8% at 32 threads. This is directly related to the In execution efficiency from the second table, since more sched overhead means worse execution efficiency. The synch Overhead also grows with more threads, which makes sense because more threads means more chances of one thread having to wait for another. Looking at the LB metrics, the first LB goes from 1.0 to 0.36, which means some threads execute a lot more tasks than others. But the second LB stays more stable, going from 1.0 to 0.93, which tells us that even if the tasks are not evenly distributed, they are all similar in length. This second LB is directly related to the Load balancing from the second table, since it measures the difference in time between tasks.

Statistics about explicit tasks in parallel fraction										
Number of threads	1	2	4	8	12	16	20	24	28	32
Number of explicit tasks executed (total)	8448.0	8448.0	8448.0	8448.0	8448.0	8448.0	8448.0	8448.0	8448.0	8448.0
LB (number of explicit tasks executed)	1.0	0.95	0.64	0.46	0.31	0.41	0.31	0.43	0.39	0.36
LB (time executing explicit tasks)	1.0	1.0	1.0	0.99	0.98	0.98	0.97	0.96	0.95	0.93
Time per explicit task (average us)	416.77	418.36	419.25	421.71	425.35	431.03	435.64	438.18	447.34	454.92
Overhead per explicit task (synch %)	0.0	0.36	0.66	1.36	2.67	4.36	5.03	5.93	7.62	8.71
Overhead per explicit task (sched %)	0.05	0.18	0.36	0.88	1.49	2.57	3.4	4.12	5.99	7.8
Number of taskwait/taskgroup (total)	64.0	64.0	64.0	64.0	64.0	64.0	64.0	64.0	64.0	64.0

Paraver analysis

Hint Useful/Instantaneous Parallelism: This graph shows that at the beginning the parallelism changes a lot, which means threads are starting and picking up tasks at different times. After that it becomes more stable and most threads are working at the same time. However at the end we can see it drops, which means some threads finish before others and stay idle. This confirms the load imbalance we saw in the tables.

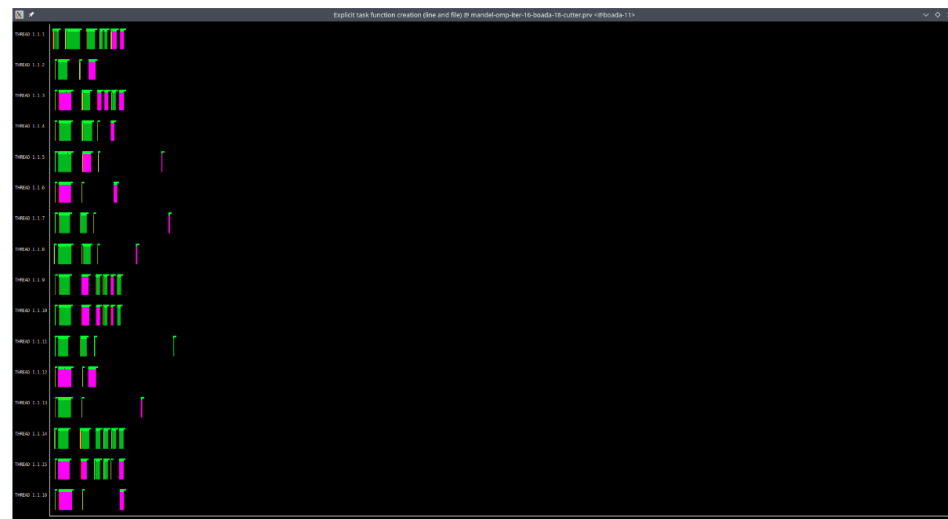


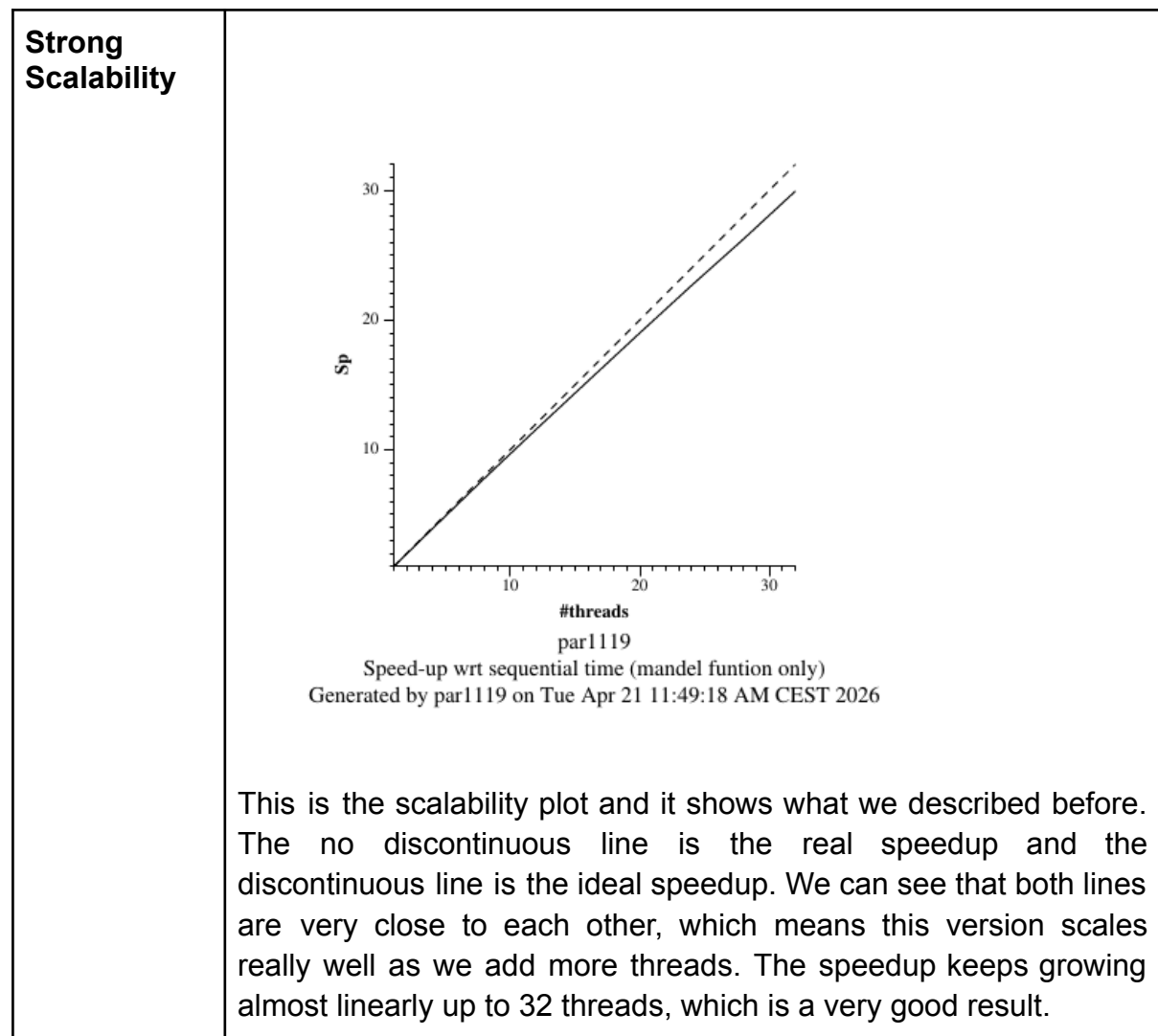
Hint Tasking/Explicit task functions executed: This diagram shows that all threads start executing tasks at the beginning and then keep working for a long time. However we can see that some

threads work for longer than others, which again confirms the load imbalance we saw in the tables. This happens because some tasks are faster to compute than others, so some threads finish earlier and stay idle while others are still working.



Hint Tasking/Explicit task functions created: This diagram shows that the green and pink blocks appear mostly at the beginning for all threads, which means all the tasks are created at the start. After that the screen goes mostly black, which means no more tasks are being created. This is good because it means the master thread creates everything early and the other threads can just focus on executing the work.





Leaf task decomposition strategy

Section	Description
Task Decomposition	Leaf task decomposition (normal)
Code	All the code can be found at “mandel-omp-leaf-normal.c”. Basically we have applied the standard leaf task decomposition to the function <i>mandel_tiled_rec</i>. We also had to put the <i>omp parallel</i> and <i>omp single</i> instructions before the first call. Tasks for the base cases We added <code>#pragma omp task</code> to both of the base cases, with their correspondent <i>critical</i> or <i>atomic</i> if needed. There is no need to make a <code>firstprivate(x, y)</code> because <i>x</i> and <i>y</i> are local variables from the function, and therefore, each thread will have

	its own copy of x and y in the stack.																																																																																																																																																																					
Modelfactor analysis	• First table: This table tells us that the elapsed time does not change after 2 or more threads, so the optimum number of threads for the code is 2 (which is terrible). The speed up suffers from the same thing, it does not change after using 2 or more threads. The consequences of both things can be observed in the efficiency, that reaches the mark of 0.04 when using 32 threads, this means that the code is not taking profit from the extra threads. <table><tr><th colspan="11">Overview of whole program execution metrics</th></tr><tr><th>Number of threads</th><th>1</th><th>2</th><th>4</th><th>8</th><th>12</th><th>16</th><th>20</th><th>24</th><th>28</th><th>32</th></tr><tr><td>Elapsed time (sec)</td><td>1.83</td><td>1.41</td><td>1.41</td><td>1.41</td><td>1.41</td><td>1.41</td><td>1.41</td><td>1.41</td><td>1.41</td><td>1.41</td></tr><tr><td>Speedup</td><td>1.00</td><td>1.30</td><td>1.30</td><td>1.30</td><td>1.30</td><td>1.30</td><td>1.30</td><td>1.30</td><td>1.29</td><td>1.29</td></tr><tr><td>Efficiency</td><td>1.00</td><td>0.65</td><td>0.32</td><td>0.16</td><td>0.11</td><td>0.08</td><td>0.06</td><td>0.05</td><td>0.05</td><td>0.04</td></tr></table> Table 1: Analysis done on Tue May 12 03:21:24 PM CEST 2026, par1104 • Second table: To see what is happening lets check the second table. The parallel fraction of the program is pretty big, 99.98%, so this is not the problem. The <i>Global efficiency</i> from the parallel region reaches the mark of 4.04%, which means that something is going bad. The <i>Scalability for computing tasks</i> is really high at the end, so this is not what is causing the poor performance. The <i>Parallelization strategy efficiency</i> reaches the mark of 4.13%. When analyzing the <i>In execution efficiency</i> we can see that this is not the problem. So the last option left is the <i>Load balancing</i>, and when seeing its mark of 4.15% we can say without hesitation that this is the problem. This Load balancing mark says that there is a big difference between the average time that a thread works with the maximum time that a thread reaches at some point. This is directly related to efficiency, when one decreases the other one also does. <table><tr><th colspan="11">Overview of the Efficiency metrics in parallel fraction, $\phi=99.98\%$</th></tr><tr><th>Number of threads</th><th>1</th><th>2</th><th>4</th><th>8</th><th>12</th><th>16</th><th>20</th><th>24</th><th>28</th><th>32</th></tr><tr><td>Global efficiency</td><td>99.96%</td><td>64.91%</td><td>32.43%</td><td>16.20%</td><td>10.80%</td><td>8.10%</td><td>6.47%</td><td>5.40%</td><td>4.62%</td><td>4.04%</td></tr><tr><td>Parallelization strategy efficiency</td><td>99.96%</td><td>65.01%</td><td>32.52%</td><td>16.27%</td><td>10.87%</td><td>8.16%</td><td>6.54%</td><td>5.47%</td><td>4.70%</td><td>4.13%</td></tr><tr><td>Load balancing</td><td>100.00%</td><td>65.17%</td><td>32.63%</td><td>16.33%</td><td>10.91%</td><td>8.19%</td><td>6.57%</td><td>5.49%</td><td>4.72%</td><td>4.15%</td></tr><tr><td>In execution efficiency</td><td>99.96%</td><td>99.74%</td><td>99.66%</td><td>99.61%</td><td>99.63%</td><td>99.61%</td><td>99.61%</td><td>99.59%</td><td>99.58%</td><td>99.56%</td></tr><tr><td>Scalability for computation tasks</td><td>100.00%</td><td>99.85%</td><td>99.71%</td><td>99.59%</td><td>99.34%</td><td>99.21%</td><td>98.95%</td><td>98.62%</td><td>98.26%</td><td>97.86%</td></tr><tr><td>IPC scalability</td><td>100.00%</td><td>99.86%</td><td>99.78%</td><td>99.74%</td><td>99.74%</td><td>99.73%</td><td>99.69%</td><td>99.71%</td><td>99.70%</td><td>99.73%</td></tr><tr><td>Instruction scalability</td><td>100.00%</td><td>100.07%</td><td>100.07%</td><td>100.07%</td><td>100.06%</td><td>100.06%</td><td>100.06%</td><td>100.06%</td><td>100.06%</td><td>100.06%</td></tr><tr><td>Frequency scalability</td><td>100.00%</td><td>99.93%</td><td>99.87%</td><td>99.78%</td><td>99.54%</td><td>99.42%</td><td>99.20%</td><td>98.86%</td><td>98.50%</td><td>98.07%</td></tr></table> Table 2: Analysis done on Tue May 12 03:21:24 PM CEST 2026, par1104 • Third table: When seeing the table the first thing we notice is that the <i>synch Overhead</i> reaches a point of 10090, and this means that for every 100s of computing we use 10090s to synchronize, and that is terrible, we spend more time synchronizing threads than	Overview of whole program execution metrics											Number of threads	1	2	4	8	12	16	20	24	28	32	Elapsed time (sec)	1.83	1.41	1.41	1.41	1.41	1.41	1.41	1.41	1.41	1.41	Speedup	1.00	1.30	1.30	1.30	1.30	1.30	1.30	1.30	1.29	1.29	Efficiency	1.00	0.65	0.32	0.16	0.11	0.08	0.06	0.05	0.05	0.04	Overview of the Efficiency metrics in parallel fraction, $\phi=99.98\%$											Number of threads	1	2	4	8	12	16	20	24	28	32	Global efficiency	99.96%	64.91%	32.43%	16.20%	10.80%	8.10%	6.47%	5.40%	4.62%	4.04%	Parallelization strategy efficiency	99.96%	65.01%	32.52%	16.27%	10.87%	8.16%	6.54%	5.47%	4.70%	4.13%	Load balancing	100.00%	65.17%	32.63%	16.33%	10.91%	8.19%	6.57%	5.49%	4.72%	4.15%	In execution efficiency	99.96%	99.74%	99.66%	99.61%	99.63%	99.61%	99.61%	99.59%	99.58%	99.56%	Scalability for computation tasks	100.00%	99.85%	99.71%	99.59%	99.34%	99.21%	98.95%	98.62%	98.26%	97.86%	IPC scalability	100.00%	99.86%	99.78%	99.74%	99.74%	99.73%	99.69%	99.71%	99.70%	99.73%	Instruction scalability	100.00%	100.07%	100.07%	100.07%	100.06%	100.06%	100.06%	100.06%	100.06%	100.06%	Frequency scalability	100.00%	99.93%	99.87%	99.78%	99.54%	99.42%	99.20%	98.86%	98.50%	98.07%
	Overview of whole program execution metrics																																																																																																																																																																					
	Number of threads	1	2	4	8	12	16	20	24	28	32																																																																																																																																																											
	Elapsed time (sec)	1.83	1.41	1.41	1.41	1.41	1.41	1.41	1.41	1.41	1.41																																																																																																																																																											
	Speedup	1.00	1.30	1.30	1.30	1.30	1.30	1.30	1.30	1.29	1.29																																																																																																																																																											
	Efficiency	1.00	0.65	0.32	0.16	0.11	0.08	0.06	0.05	0.05	0.04																																																																																																																																																											
	Overview of the Efficiency metrics in parallel fraction, $\phi=99.98\%$																																																																																																																																																																					
	Number of threads	1	2	4	8	12	16	20	24	28	32																																																																																																																																																											
	Global efficiency	99.96%	64.91%	32.43%	16.20%	10.80%	8.10%	6.47%	5.40%	4.62%	4.04%																																																																																																																																																											
	Parallelization strategy efficiency	99.96%	65.01%	32.52%	16.27%	10.87%	8.16%	6.54%	5.47%	4.70%	4.13%																																																																																																																																																											
Load balancing	100.00%	65.17%	32.63%	16.33%	10.91%	8.19%	6.57%	5.49%	4.72%	4.15%																																																																																																																																																												
In execution efficiency	99.96%	99.74%	99.66%	99.61%	99.63%	99.61%	99.61%	99.59%	99.58%	99.56%																																																																																																																																																												
Scalability for computation tasks	100.00%	99.85%	99.71%	99.59%	99.34%	99.21%	98.95%	98.62%	98.26%	97.86%																																																																																																																																																												
IPC scalability	100.00%	99.86%	99.78%	99.74%	99.74%	99.73%	99.69%	99.71%	99.70%	99.73%																																																																																																																																																												
Instruction scalability	100.00%	100.07%	100.07%	100.07%	100.06%	100.06%	100.06%	100.06%	100.06%	100.06%																																																																																																																																																												
Frequency scalability	100.00%	99.93%	99.87%	99.78%	99.54%	99.42%	99.20%	98.86%	98.50%	98.07%																																																																																																																																																												

making the current work.

Both LB have a low mark, so they are not a huge problem. The *sched Overhead* is really low, and does not impact on the execution time.

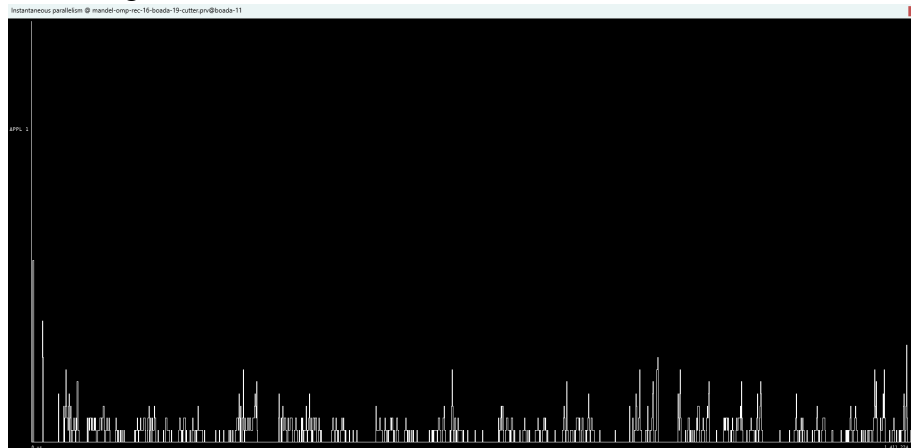
Statistics about explicit tasks in parallel fraction										
Number of threads	1	2	4	8	12	16	20	24	28	32
Number of explicit tasks executed (total)	3013.0	3013.0	3013.0	3013.0	3013.0	3013.0	3013.0	3013.0	3013.0	3013.0
LB (number of explicit tasks executed)	1.0	0.54	0.64	0.88	0.83	0.78	0.8	0.74	0.75	0.78
LB (time executing explicit tasks)	1.0	0.5	0.71	0.72	0.61	0.57	0.59	0.62	0.45	0.57
Time per explicit task (average us)	141.27	141.83	142.06	142.16	142.11	142.16	142.06	142.1	142.25	142.33
Overhead per explicit task (synch %)	0.0	229.79	887.21	2202.78	3519.27	4833.91	6157.68	7468.72	8779.74	10090.95
Overhead per explicit task (sched %)	0.18	0.66	0.76	0.83	0.76	0.76	0.78	0.79	0.79	0.86
Number of taskwait/taskgroup (total)	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

Table 3: Analysis done on Tue May 12 03:21:24 PM CEST 2026, par1104

Paraver analysis

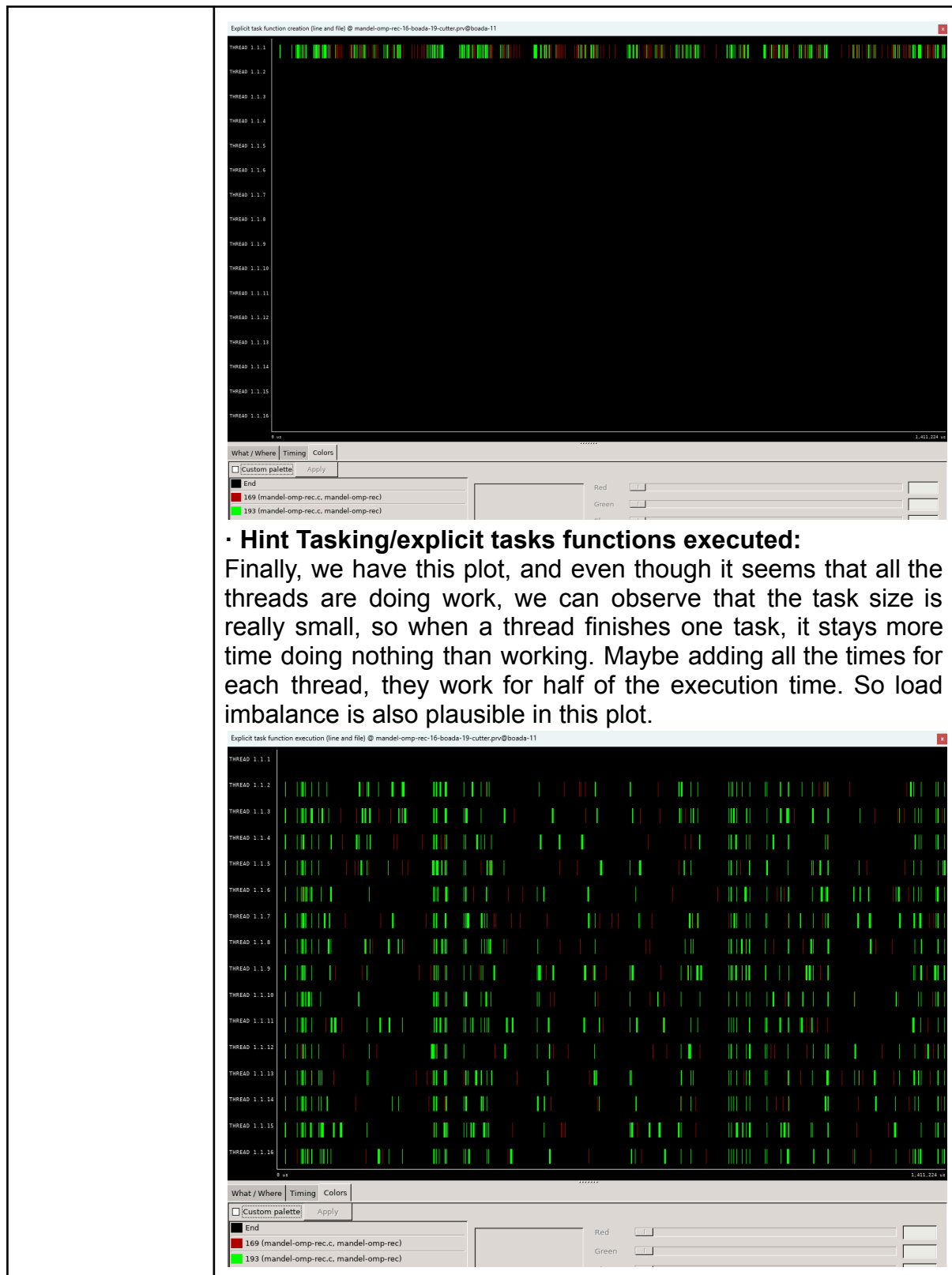
• Hint Useful/Instantaneous Parallelism:

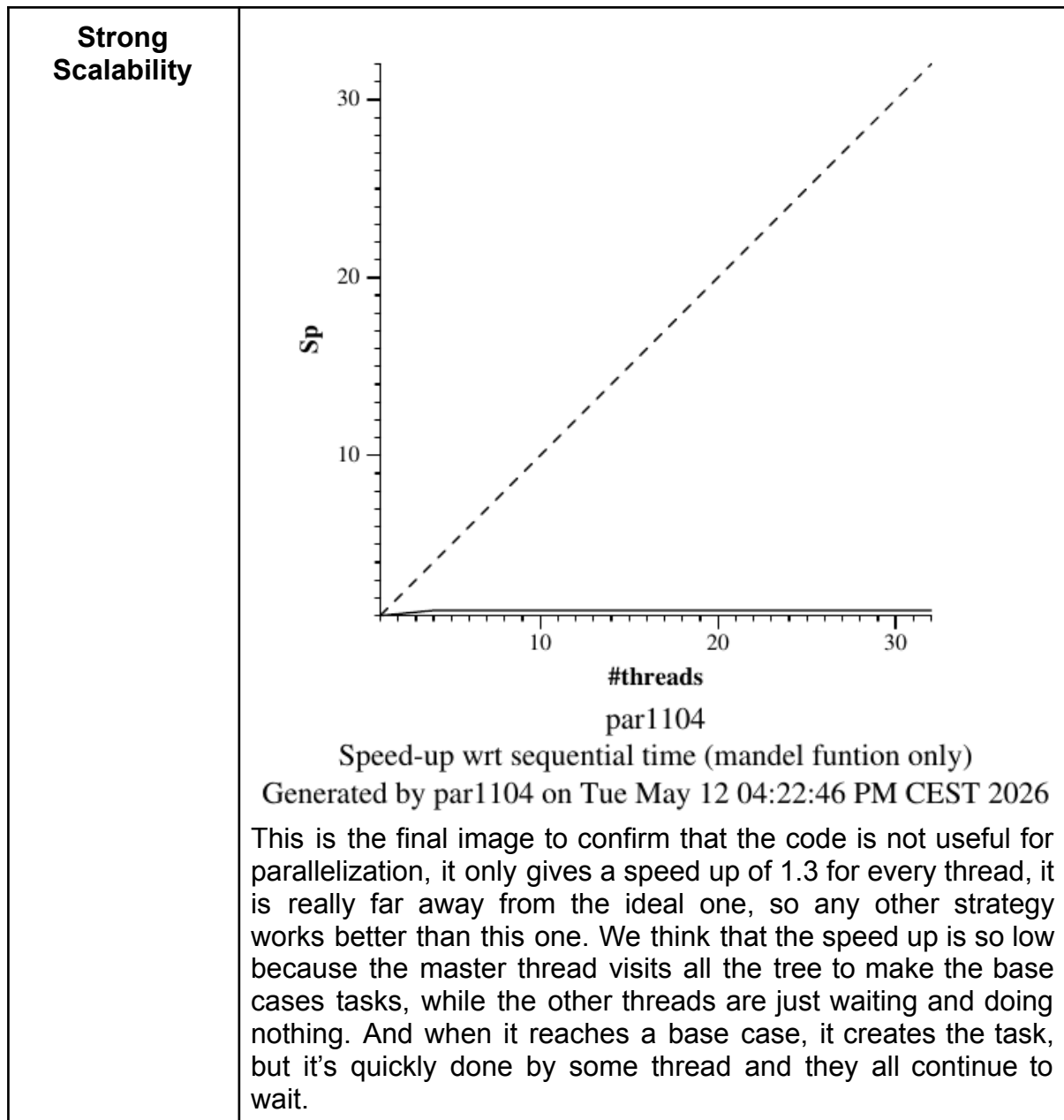
The plot shows that there is not a profit about parallelism, because there is not a constant use of it, it makes peaks and then it drops. This is a bad sign related to the poor load balancing that we have talked about before.



• Hint Tasking/explicit tasks functions created:

This one is normal, and shows us that the tasks are created by the master thread with the single instruction. We can observe that more tasks of the second case base are being created.





Section	Description
Task Decomposition	Leaf task decomposition (improved)
Code	The full code can be found at the file "mandel-omp-leaf-millorat.c". In this section we will cover just the most important details for the leaf task decomposition with cut-off. With this strategy we want to increase the granularity of the tasks to obtain a better speed-up, because in the case of not doing it, the load balance would be really bad.

	Depth Control With this strategy we need to determine at which depth the base cases stop being “base cases”. In our code the most optimal depth is <code>MAXDEPTH = 3</code>, and this means that when <code>d <= 3</code> tasks are created in the base cases, but, when <code>d == 3</code> and we enter in the recursive cases, we just create a one big task that englobes all the recursive cases, next to that, no more tasks will be created (because <code>d >= 4</code> and the conditions won't permit the creation of more tasks). Implementation To implement this features we put in the base case headers: <pre>#pragma omp task if (d <= MAXDEPTH)</pre> And we also put the <i>atomic</i> and the <i>critical</i> as we did in the previous codes to reduce possible overheads by creating the image and the histogram. We don't put the <code>firstprivate(x, y)</code> following the same reason as before. For the recursive cases we just did the following: <pre>if (d == MAXDEPTH) #pragma omp task { *recursive calls } else *recursive calls as usual</pre>
Modelfactor analysis	• First table: Now, we can see from this table that the elapsed time starts to decrease thanks to parallelization, but it remains constant when it reaches 8 threads. This happens because the speedup also remains constant when it arrives at that number of threads. The mix of these two indicators significantly reduces efficiency, because even though the number of threads increases, the elapsed time does not. We have to say that it's a big improvement coming from the normal leaf task decomposition, it is three times quicker and reaches a speed up of 4.16, which is four times greater than the 1.3.

Overview of whole program execution metrics										
Number of threads	1	2	4	8	12	16	20	24	28	32
Elapsed time (sec)	1.83	1.02	0.58	0.44	0.44	0.44	0.44	0.44	0.44	0.44
Speedup	1.00	1.79	3.14	4.17	4.17	4.17	4.16	4.16	4.17	4.16
Efficiency	1.00	0.90	0.78	0.52	0.35	0.26	0.21	0.17	0.15	0.13

Table 1: Analysis done on Tue May 5 11:06:23 AM CEST 2026, par1104

• **Second table:**

In the second table, we can observe that the parallel fraction of the program is 99.98%, which is good because that means that almost every part of the code is parallelizable. Besides that, the *Global efficiency* (of the parallel fraction) is terrible when it reaches 32 threads.

If we start to investigate, we can observe that the *Scalability for computation tasks* is really good, so that is not the cause of the problems. So the only option left is the *Parallelization strategy efficiency*, and this is the problem, because when it reaches 32 threads the parallelization is only about 13.25%.

Even though the *In execution efficiency* is not the best, the biggest trouble can be found in the *Load balancing*. This metric reaches the poor mark of 17.82%, and with this result, we can end up with the conclusion that there is a big difference between the average time from a thread and the maximum time that a thread reaches. This can be directly related with the poor efficiency that we saw in the first table, because poorer load balancing worse efficiency we will obtain. Comparing the results with the previous strategy, we see the same thing, that the load imbalance is plausible and worsens the results, but with this new strategy we obtain some better results.

Overview of the Efficiency metrics in parallel fraction, $\phi=99.98\%$										
Number of threads	1	2	4	8	12	16	20	24	28	32
Global efficiency	99.96%	89.57%	78.37%	52.12%	34.77%	26.06%	20.81%	17.35%	14.88%	13.01%
Parallelization strategy efficiency	99.96%	89.65%	78.49%	52.27%	34.89%	26.21%	20.99%	17.55%	15.10%	13.25%
Load balancing	100.00%	89.72%	86.56%	70.24%	46.91%	35.23%	28.27%	23.62%	20.30%	17.82%
In execution efficiency	99.96%	99.92%	90.68%	74.41%	74.38%	74.41%	74.25%	74.29%	74.37%	74.33%
Scalability for computation tasks	100.00%	99.90%	99.84%	99.71%	99.65%	99.40%	99.18%	98.86%	98.54%	98.19%
IPC scalability	100.00%	99.97%	99.95%	99.92%	99.91%	99.91%	99.91%	99.91%	99.90%	99.90%
Instruction scalability	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
Frequency scalability	100.00%	99.93%	99.89%	99.79%	99.74%	99.49%	99.26%	98.95%	98.65%	98.28%

• **Third table:**

The last table tells us more about the load imbalance, because the *synch Overhead* grows more and more as we use more threads, and reaches the horrible mark of 2538%. This means that for every 100s of computing, we waste 2538s with overheads, and that is the cause of the poor efficiency. This is a huge reduction coming from the normal strategy, and enables better time marks.

Next to that, the *sched Overhead* when reaching 8 threads or more the mark stays steady around 0.35, this can be related to the poor load balance, and that a lot of threads stay in idle state and doing nothing.

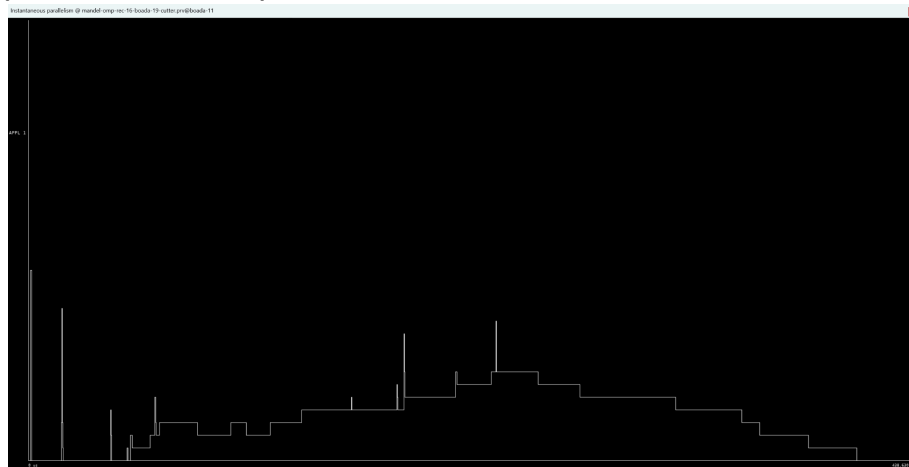
As usual, both LB decreases when increasing the number of threads, and this does not have to imply anything bad at all.

Statistics about explicit tasks in parallel fraction										
Number of threads	1	2	4	8	12	16	20	24	28	32
Number of explicit tasks executed (total)	3040.0	3040.0	3040.0	3040.0	3040.0	3040.0	3040.0	3040.0	3040.0	3040.0
LB (number of explicit tasks executed)	1.0	0.9	0.64	0.67	0.42	0.44	0.3	0.38	0.26	0.22
LB (time executing explicit tasks)	1.0	0.96	0.78	0.7	0.47	0.37	0.29	0.25	0.24	0.21
Time per explicit task (average us)	156.69	156.93	157.1	157.17	157.24	157.33	157.27	157.27	157.32	157.3
Overhead per explicit task (synch %)	0.0	43.92	104.63	348.45	714.3	1079.17	1444.87	1809.11	2172.67	2538.08
Overhead per explicit task (sched %)	0.16	0.22	0.23	0.31	0.35	0.35	0.35	0.32	0.37	0.35
Number of taskwait/taskgroup (total)	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

Paraver analysis

• Hint Useful/Instantaneous Parallelism:

This first diagram shows us that the main parallelization is at the middle of the execution, when all the tasks have been created and the threads can work in parallel. Then, when these are done they don't produce any more tasks, so the thread will stay idle and the parallelization drops, as we can see in the figure. This is what is causing the load imbalance that we have seen all the time. But this plot is more consistent and better than the previous one, the parallelization is more constant.



• Hint Tasking/explicit tasks functions created:

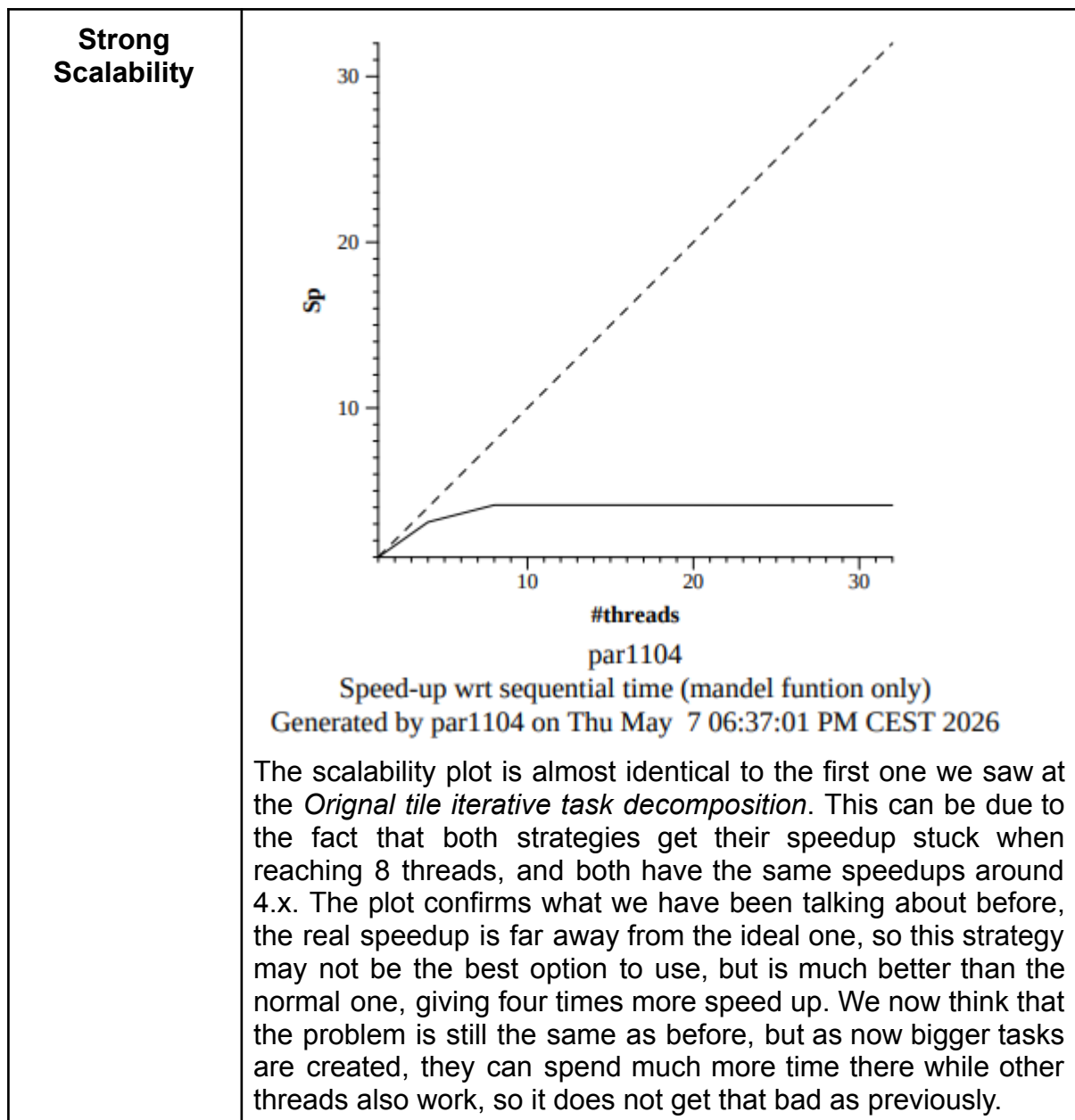
This plot has a similar shape like the previous one, indicating that the task creation is directly related to the parallelization. At the beginning just a few tasks are created, then at the middle of the execution a lot of them are created, finally few and few emerge, leaving a lot of threads in idle state.



• **Hint Tasking/explicit tasks functions executed:**

Again, this plot is similar in shape (like a mountain) to the previous one, this can tell us that in a certain point when a task is created it is directly executed and then none is created.





Tree Decomposition task

Section	Description
Task Decomposition	Tree Decomposition task
Code	The full code is in "mandel-omp-tree.c". We are going to explain it in three parts. Border computation Like in the fine grain version, we use a <code>#pragma omp taskgroup</code>

	task_reduction(&:equal). Inside the taskgroup we create two tasks with in_reduction(&:equal): • The first task computes the vertical borders • The second task computes the horizontal borders. We split them into two separate tasks so both borders can be computed at the same time. The in_reduction is needed so each task can safely update the shared equal variable without causing a data race condition. If(equal) We use: • #pragma omp atomic to protect the histogram update, because it is a simple arithmetic operation on a shared array and we need to avoid a data race condition. • #pragma omp critical to protect the XSetForeground and XDrawPoint calls, because they are function calls that need to run as a complete block, so atomic is not enough. We apply the same atomic and critical in the else branch, it's very similar than the fine grain. Recursive Mode (Tree) We divide it and call the function again. Each recursive call has a #pragma omp task, so all the subcalls can run in parallel. Depend of the NRows and TILE we create 4 tasks or we create 2 task to run This creates a tree of tasks that grows as the recursion goes deeper.
Modelfactor analysis	First table: The elapsed time keeps going down as we add more threads, from 1.84 seconds with 1 thread all the way to 0.10 seconds with 32 threads. Because of this, the speedup grows well and reaches 17.76 with 32 threads. The efficiency drops as threads increase, which is expected because efficiency is calculated as Speedup / Number of threads, so even if the speedup grows, adding more threads makes the efficiency go down. So if the SpeedUp doesn't increase a lot but the thread increase more than speedup we can see the efficiency go down.

Overview of whole program execution metrics										
Number of threads	1	2	4	8	12	16	20	24	28	32
Elapsed time (sec)	1.84	0.93	0.48	0.26	0.19	0.15	0.13	0.12	0.11	0.10
Speedup	1.00	1.97	3.84	7.05	9.78	12.00	13.81	15.40	16.71	17.76
Efficiency	1.00	0.99	0.96	0.88	0.82	0.75	0.69	0.64	0.60	0.55

Table 1: Analysis done on Tue May 5 10:12:39 AM CEST 2026, par1119

Second Table: The global efficiency drops from 99.72% to 55.48% with 32 threads. This is calculated as scalability $\times$ parallelization strategy efficiency. The scalability for computation tasks stays very high at 97.73%, so it barely affects the result. The main reason for the drop is the parallelization strategy efficiency, which falls from 99.72% to 56.77%. This metric is calculated as load balancing $\times$ in execution efficiency. The load balancing drops to 76.05% with 32 threads, and this is directly caused by the synch overhead growing, because when tasks are not evenly distributed, many threads finish early and have to wait for the slowest one, and all that waiting time pulls the load balancing down. The in execution efficiency also drops to 74.65%, and this is directly caused by the sched overhead growing, because as more threads need to synchronize and combine their results from the task_reduction, threads spend more time on scheduling and less time actually computing. Since both load balancing and in execution efficiency drop together, the parallelization strategy efficiency drops a lot, and with it the global efficiency. The IPC, instruction and frequency scalability all stay very close to 100%, which means that when threads are actually computing, the hardware is being used well.

Overview of the Efficiency metrics in parallel fraction, $\phi=99.98\%$										
Number of threads	1	2	4	8	12	16	20	24	28	32
Global efficiency	99.72%	98.25%	95.83%	87.98%	81.39%	74.93%	69.00%	64.10%	59.68%	55.48%
Parallelization strategy efficiency	99.72%	98.28%	95.93%	88.20%	81.89%	75.65%	69.85%	65.11%	60.85%	56.77%
Load balancing	100.00%	99.08%	96.77%	95.26%	92.94%	88.07%	87.10%	81.59%	81.96%	76.05%
In execution efficiency	99.72%	99.19%	99.13%	92.59%	88.12%	85.90%	80.19%	79.80%	74.24%	74.65%
Scalability for computation tasks	100.00%	99.97%	99.90%	99.74%	99.39%	99.05%	98.79%	98.45%	98.08%	97.73%
IPC scalability	100.00%	99.84%	99.81%	99.66%	99.53%	99.36%	99.35%	99.32%	99.27%	99.28%
Instruction scalability	100.00%	100.23%	100.22%	100.22%	100.22%	100.21%	100.21%	100.20%	100.20%	100.19%
Frequency scalability	100.00%	99.90%	99.87%	99.86%	99.64%	99.48%	99.23%	98.92%	98.60%	98.25%

Table 2: Analysis done on Tue May 5 10:12:39 AM CEST 2026, par1119

Third Table: The total number of tasks is always 12050 regardless of the number of threads, which means the recursion always generates the same number of tasks. The number of taskwait/taskgroup is always 4017, which corresponds to all the

taskgroups created during the recursion for computing the borders. The sched overhead is not 0 because we use task_reduction and in_reduction for the equal variable, so tasks need to synchronize and combine their results when they finish, which introduces scheduling overhead. This overhead grows from 0.19% with 1 thread to 8.36% with 32 threads, and it is directly related to the drop in “in execution efficiency” from the second table. The synch overhead also grows a lot, from 0.18% to 77.14% with 32 threads, and it is directly related to the drop in load balancing from the second table, because when tasks are not evenly distributed, many threads finish early and have to wait for the slowest one. Looking at the LB metrics, the first LB drops from 1.0 to 0.6, which means some threads execute more tasks than others. The second LB drops from 1.0 to 0.74, which tells us that tasks also differ in length, with some taking more time to compute than others. The difference than fine grain it's this create more tasks and more taskwait and taskgroup because it's recursive and doesn't have cut off.

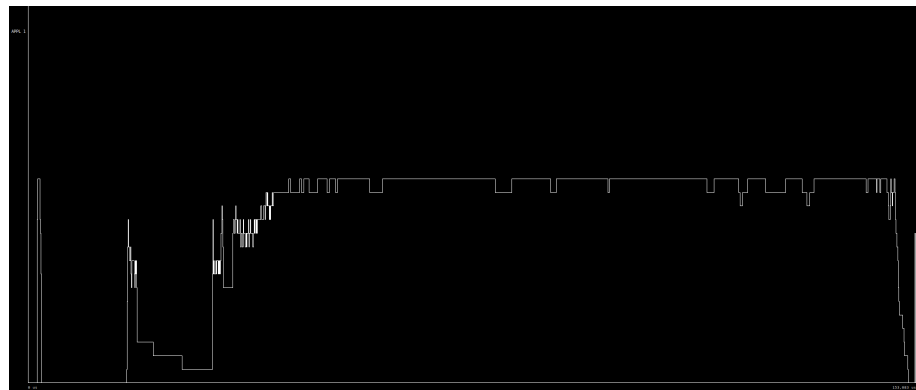
Statistics about explicit tasks in parallel fraction										
Number of threads	1	2	4	8	12	16	20	24	28	32
Number of explicit tasks executed (total)	12050.0	12050.0	12050.0	12050.0	12050.0	12050.0	12050.0	12050.0	12050.0	12050.0
LB (number of explicit tasks executed)	1.0	0.99	0.47	0.63	0.42	0.5	0.49	0.53	0.58	0.6
LB (time executing explicit tasks)	1.0	0.99	0.92	0.84	0.86	0.8	0.81	0.77	0.7	0.74
Time per explicit task (average us)	116.86	117.34	117.44	119.05	120.37	123.69	126.85	126.61	130.87	132.11
Overhead per explicit task (synch %)	0.18	1.99	5.07	16.42	25.93	35.99	46.09	58.31	66.1	77.14
Overhead per explicit task (sched %)	0.19	0.24	0.29	0.47	1.37	2.38	3.99	4.94	6.57	8.36
Number of taskwait/taskgroup (total)	4017.0	4017.0	4017.0	4017.0	4017.0	4017.0	4017.0	4017.0	4017.0	4017.0

Table 3: Analysis done on Tue May 5 10:12:39 AM CEST 2026, par1119

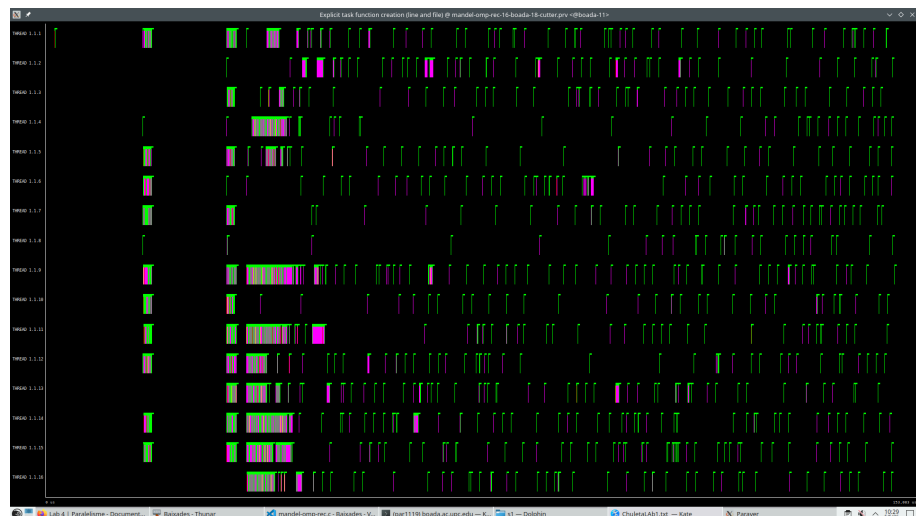
Paraver analysis

Hint Useful/Instantaneous Parallelism: At the beginning we can see a stepped increase in parallelism. The first step is caused by the creation of the parallel region and the single. After that, the parallelism keeps growing slowly because the recursion works like a tree. It starts with one task that creates 4 smaller tasks, and each of those creates 4 more, and so on. This means that at the start there are very few tasks available, so most threads are waiting with nothing to do. As more and more tasks are created, more threads can work at the same time and the parallelism goes up. Once enough tasks have been created, the parallelism becomes stable and most threads are working at the same time. At the end we can see a drop, which means some threads finish before others and stay idle.

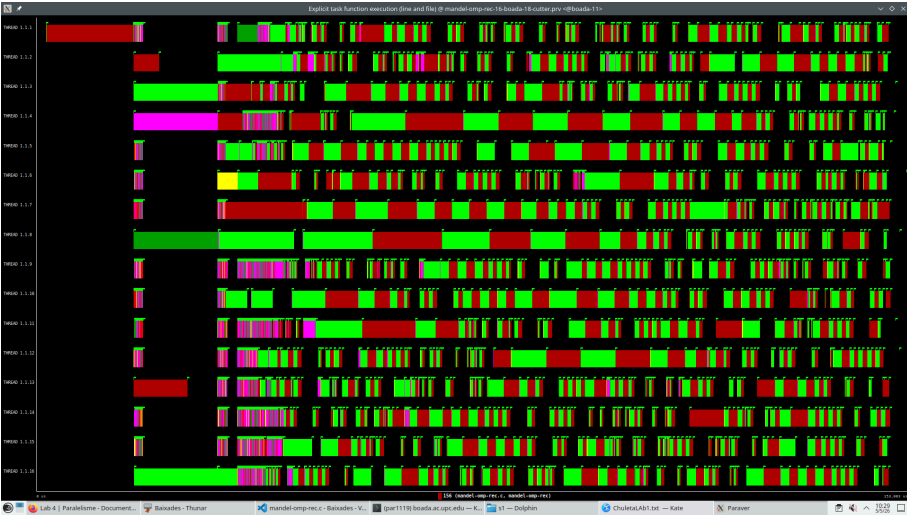
This confirms the load imbalance we saw in the tables.

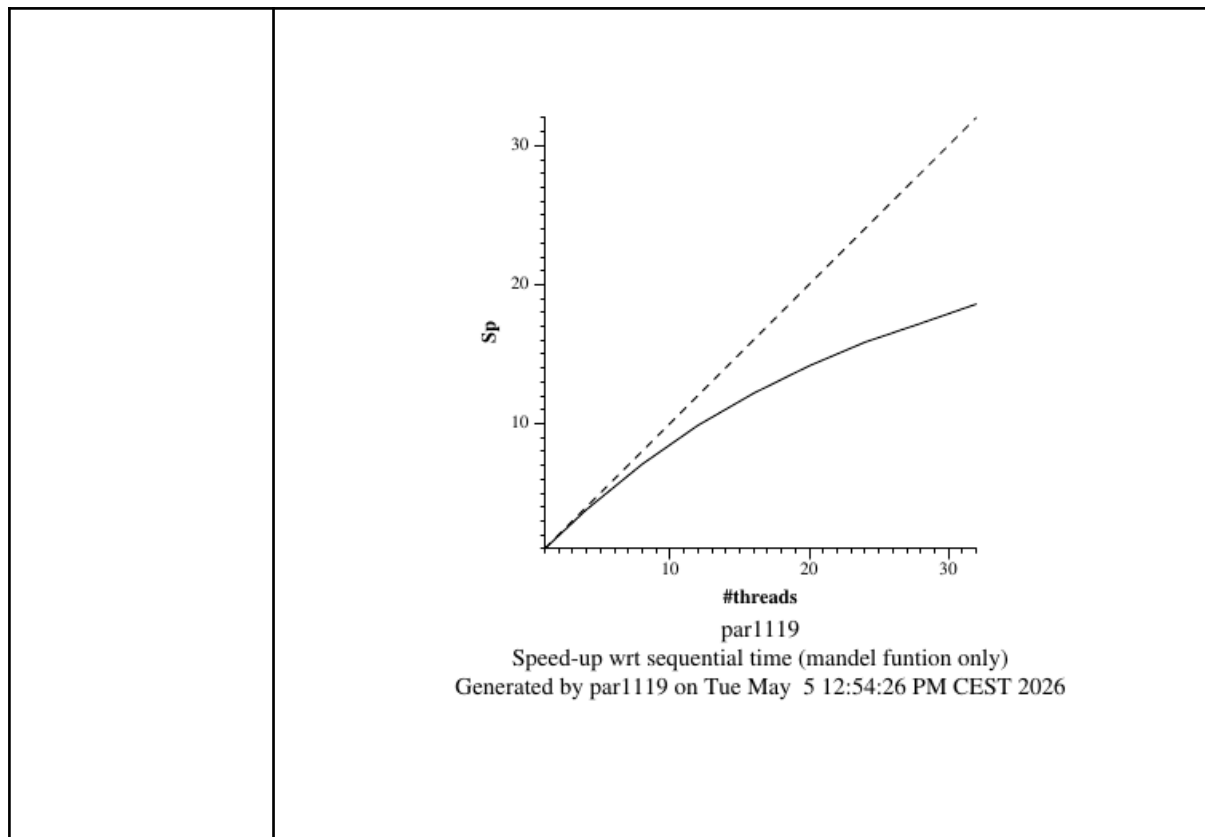


Hint Tasking/Explicit task functions created: The characteristic of this version is that tasks are created during the whole execution and by all threads, because the recursion keeps generating new subtasks as the tree grows deeper. This is different from the iterative versions, where all tasks were created at the beginning.



Hint Tasking/Explicit task functions executed: This diagram shows 6 different colors, each one corresponding to a line in the code where a task is executed. We can see that the red and light green, which correspond to the two border tasks inside the taskgroup, appear much more frequently than the others. This makes sense because these two tasks are executed at every node of the recursion tree, both at intermediate nodes and at leaf nodes, so they are always present. They also appear with a

	very similar frequency and duration because they are always created together inside the same taskgroup, one for the vertical border and one for the horizontal border. The other colors, which correspond to the 4 recursive calls, appear less frequently because they are only executed when the calls splits, which does not happen at every node of the tree. All threads are working during the whole execution, but we can see that some threads have more gaps than others, which again confirms the load imbalance we saw in the tables. 
Strong Scalability	Strong Scability: The discontinuous line is the ideal speedup and the continuous line is the real speedup. We can see that at the beginning, with a low number of threads, both lines are close to each other. However, as we add more threads the real speedup starts to fall further away from the ideal one, and it begins to flatten out. This is because of the load imbalance and the growing synch and sched overhead we saw in the tables. The speedup keeps growing up to 32 threads but it is far from the ideal, which tells us that this version scales but not as well as it could.



Elapsed time in seconds for each version:

	Number of threads					
Version	1	4	8	12	16	32
Iterative: Tile	3.52	1.04	0.74	0.73	0.73	0.73
Iterative: Fine grain	3.53	0.89	0.45	0.31	0.24	0.13
Recursive: Leaf	1.83	1.41	1.41	1.41	1.41	1.41
Recursive: Improved Leaf	1.83	0.58	0.44	0.44	0.44	0.44
Recursive: Tree	1.84	0.48	0.26	0.19	0.15	0.10
Best parallel implementation	Version	Reason why?				
	Iterative Fine grain	Although the Recursive Tree achieves a slightly better absolute time (0.10s vs 0.13s at 32 threads), the Fine grain version delivers a				

		significantly higher speedup relative to sequential (~26x vs ~18x) and better load balance (87.42% vs 76.05% at 32 threads). Since a higher speedup indicates more efficient use of the added threads, and load imbalance is directly tied to synchronization overhead, the Fine grain version is the best overall parallel implementation.
--	--	---